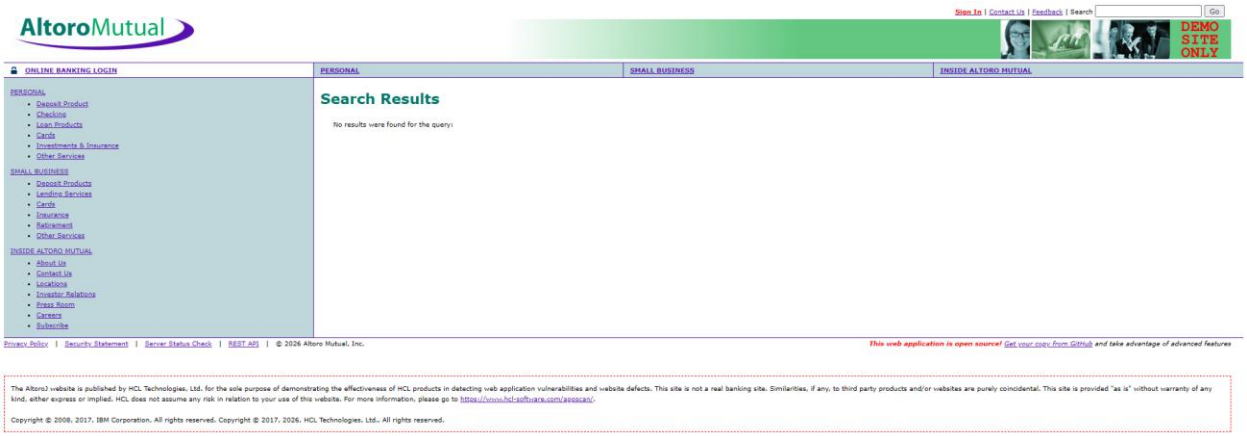
Hands-on Assessment: Cross-Site Scripting (XSS)

Objective

You are acting as a **Junior Penetration Tester**. Your task is to identify, verify, document, and recommend fixes for Cross-Site Scripting (XSS) vulnerabilities in a deliberately vulnerable web application (e.g., DVWA, OWASP Juice Shop, bWAPP, or XVWA).

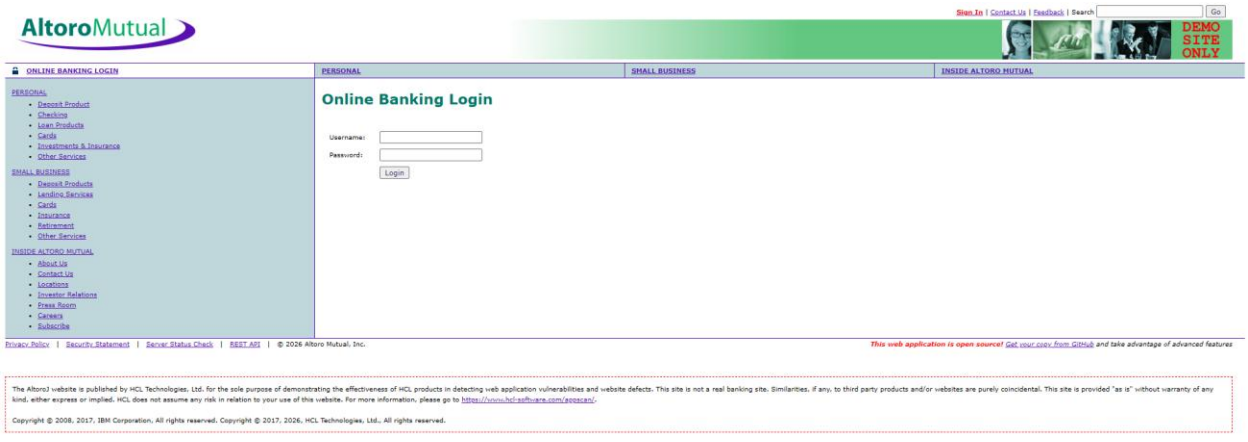
Task 1 – Identify Input Points (10 Marks)

Input Point: Search bar
URL: https://demo.testfire.net/search.jsp
Parameter: query
Screenshot:



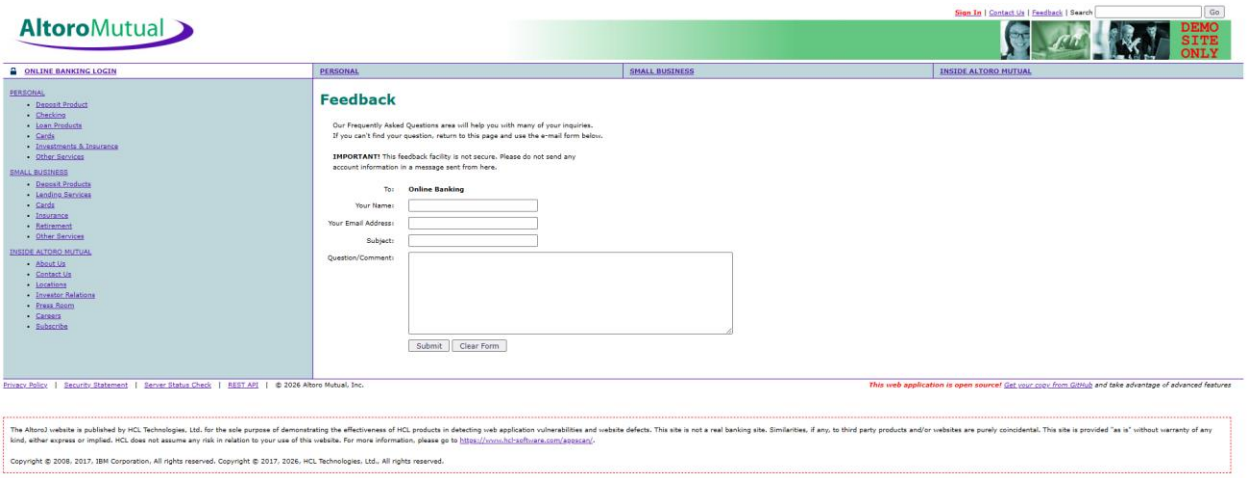
Input Point: Login Username
URL: https://demo.testfire.net/login.jsp
Parameter: uid

Input Point: Login Password
URL: https://demo.testfire.net/login.jsp
Parameter: passw
Screenshot:

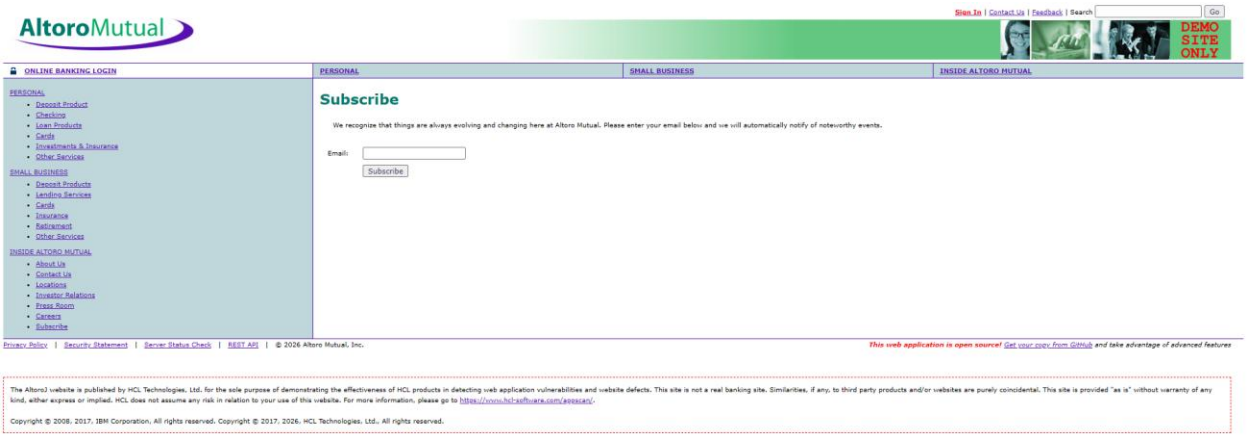


Input Point: Feedback page
URL: https://demo.testfire.net/feedback.jsp
Parameter: cfile, name, email_addr, subject, comments, submit

Screenshot:

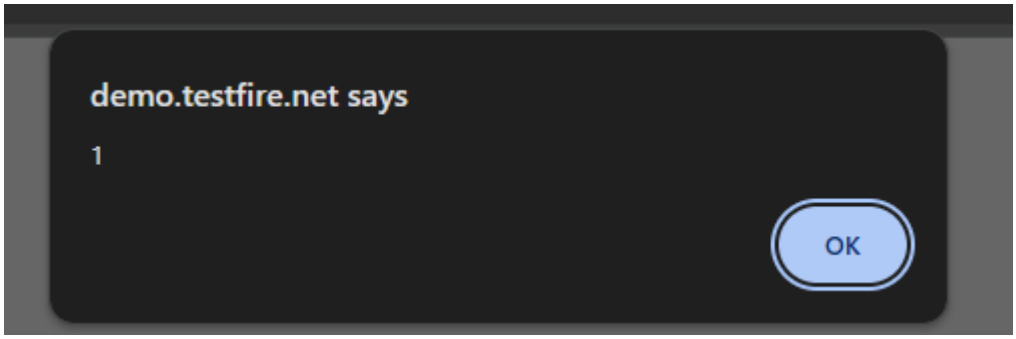


Input Point: Subscribe page
URL: https://demo.testfire.net/subscribe.jsp
Parameter: txtEmail
Screenshot:



Task 2 – Reflected XSS Testing (10 Marks)

- **Target URL**
  https://demo.testfire.net/search.jsp?query=laptop
- **Parameter tested:** query
- **Payload(s) used: query=**<script>alert(1)</script>
- **Screenshot of execution**



- **Burp Request**

Request

Pretty Raw Hex GraphQL Hackvertor

```
1  GET /search.jsp?query=<script>alert(1)</script> HTTP/1.1
2  Host: demo.testfire.net
3  Cookie: JSESSIONID=F6796BA16BA7C8F600E45B48FDBB75BF
4  Sec-Ch-Ua: "Not)A;Brand";v="8", "Chromium";v="138"
5  Sec-Ch-Ua-Mobile: ?0
6  Sec-Ch-Ua-Platform: "Windows"
7  Accept-Language: en-GB,en;q=0.9
8  Upgrade-Insecure-Requests: 1
9  User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
   Gecko) Chrome/138.0.0.0 Safari/537.36
10 Accept:
   text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;
   q=0.8,application/signed-exchange;v=b3;q=0.7
11 Sec-Fetch-Site: same-origin
12 Sec-Fetch-Mode: navigate
13 Sec-Fetch-User: ?1
14 Sec-Fetch-Dest: document
15 Referer: https://demo.testfire.net/feedback.jsp
16 Accept-Encoding: gzip, deflate, br
17 Priority: u=0, i
18 Connection: keep-alive
19
```

- **Burp Response**

Response

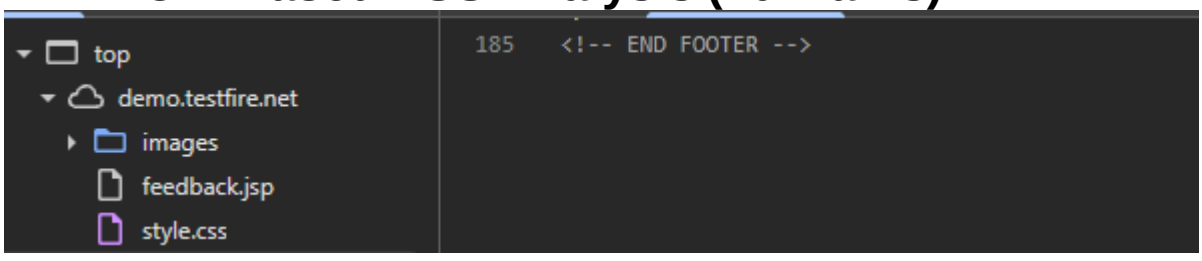
Pretty Raw Hex Render Hackvertor

```
1  HTTP/1.1 400 Bad Request
2  Server: Apache-Coyote/1.1
3  Date: Fri, 17 Jul 2026 21:14:25 GMT
4  Connection: close
5  Content-Length: 0
```

Task 3 – Stored XSS Testing (10 Marks)

- **Page tested: https://demo.testfire.net/feedback.jsp**
- **Observation:** The application accepts user input but does not redisplay submitted feedback to users. Since the input is not stored and rendered in a user-accessible location, Stored XSS could not be verified.
- **Result:** No evidence of Stored XSS was identified during testing.

Task 4 – DOM-Based XSS Analysis (10 Marks)



- **Vulnerability explanation:** The page was inspected using the browser's Developer Tools. No external or inline JavaScript was found. Since there was no client-side code reading user-controlled data from sources such as location.hash, location.search, cookies, or local storage, and no unsafe DOM manipulation APIs (such as innerHTML, document.write(), or eval()) were present, no DOM-based Cross-Site Scripting (XSS) vulnerability was identified on the tested page.

Task 5 – Manual Verification Using Burp Suite (10 Marks)

- **Burp Request**
  GET /search.jsp?query=TEST123 HTTP/1.1
  Host: demo.testfire.net

- **Burp Response**

  <h1>Search Results</h1>
  <p>No results were found for the query:

  TEST123

- **Reflection location**
  Search Results page
  The value supplied in the "query" parameter is reflected inside the HTML response.
- **Evidence**

Request

Pretty Raw Hex GraphQL Hackvertor

```
GET /search.jsp?query=TEST123 HTTP/1.1
Host: demo.testfire.net
Cookie: JSESSIONID=F6796BA16BA7C8F600E45B48FDBB75BF
Sec-Ch-Ua: "Not)A;Brand";v="8", "Chromium";v="138"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: en-GB,en;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Referer: https://demo.testfire.net/feedback.jsp
Accept-Encoding: gzip, deflate, br
Priority: u=0, i
Connection: keep-alive
```

Response

Pretty Raw Hex Render Hackvertor

```
                        <li>
                            <a id="MenuHyperLink19" href="subscribe.jsp">
                                Subscribe
                            </a>
                        </li>
                    </ul>
                </td>
                <!-- TOC END -->

                <td valign="top" colspan="3" class="bb">


                    <div class="fl" style="width: 99%;">

                        <h1>
                            Search Results
                        </h1>

                        <p>
                            No results were found for the query:<br />
                            <br />

                            TEST123
```

Inspector

Selection 7 (0x7)

Selected text: TEST123

Request attributes 2
Request query parameters 1
Request body parameters 0
Request cookies 1
Request headers 17
Response headers 4

Final Report

1. Executive Summary

A security assessment of the Altoro Mutual web application was performed to identify Cross-Site Scripting (XSS) vulnerabilities. Five input points were identified. Testing confirmed a Reflected XSS vulnerability in the search functionality through successful JavaScript execution. No Stored XSS or DOM-based XSS vulnerabilities were verified on the tested pages.

2. Scope

Application tested: Altoro Mutual (https://demo.testfire.net/)

Testing date: 17 July 2026

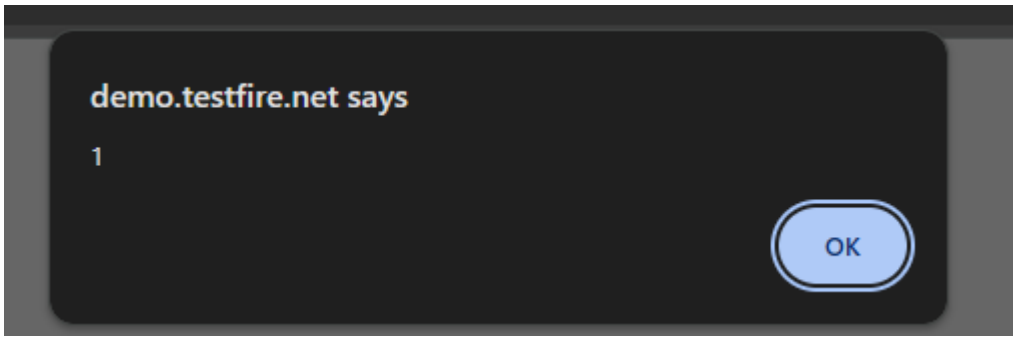
Tools used:

- **Burp Suite Professional Edition**
- **Google Chrome**
- **Browser Developer Tools**

3. Findings

For each vulnerability include:

- **Finding ID:** XSS-001
- **Vulnerability Type:** Reflected XSS
- **Affected URL:** https://demo.testfire.net/search.jsp
- **Parameter:** query
- **Severity:** Medium
- **Description:** The application reflects user input without proper output encoding, allowing JavaScript execution
- **Evidence:**

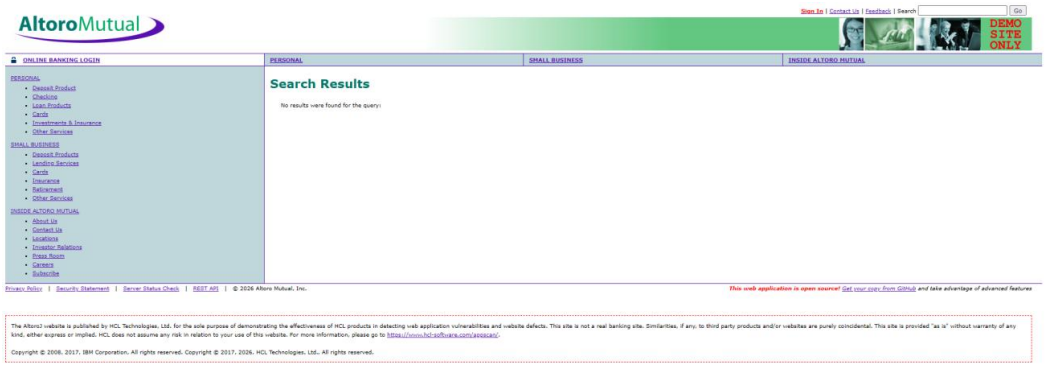


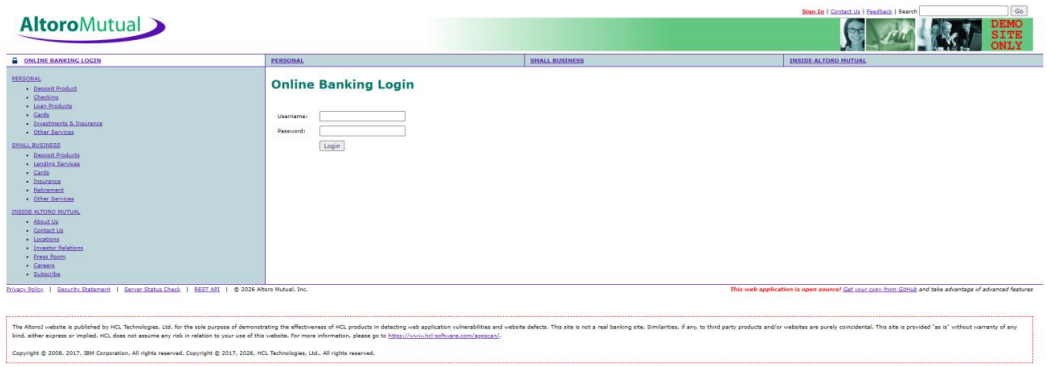
Request

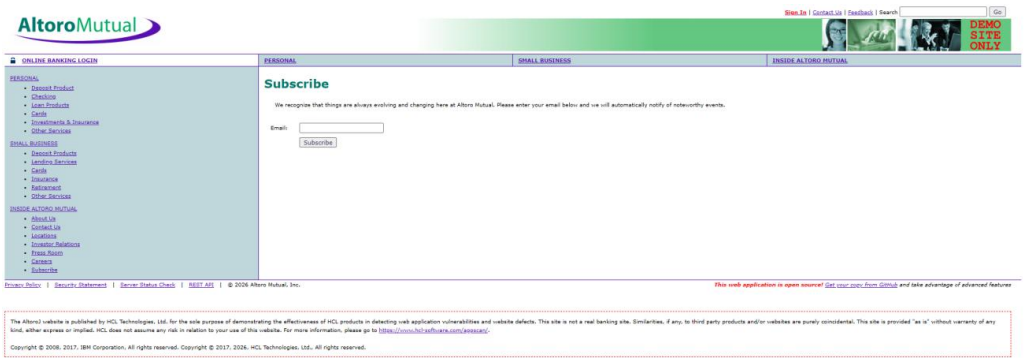
Pretty | Raw | Hex | GraphQL | Hackvertor

```
GET /search.jsp?query=<script>alert(1)</script> HTTP/1.1
Host: demo.testfire.net
Cookie: JSESSIONID=F6796BA16BA7C8F600E45B48FDBB75BF
Sec-Ch-Ua: "Not)A;Brand";v="8", "Chromium";v="138"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: en-GB,en;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Referer: https://demo.testfire.net/feedback.jsp
Accept-Encoding: gzip, deflate, br
Priority: u=0, i
Connection: keep-alive
```

- **Screenshots:**







- **Reproduction Steps:**
    - **Open the Burp Browser.**
    - **Navigate to https://demo.testfire.net/search.jsp.**
    - **Enter the following payload into the search box:**
      <script>alert(1)</script>
    - **Submit the search request.**
    - **Observe that the application reflects the input in the search results.**
    - **The browser executes the injected JavaScript and displays an alert dialog containing 1.**
    - **Capture the HTTP request and response in Burp Suite as supporting evidence.**
- **Business Impact:** Attackers may execute arbitrary JavaScript in victims' browsers, leading to phishing, session theft, or unauthorized actions.
- **Remediation:** Apply output encoding, validate input, implement CSP, avoid unsafe DOM APIs.

4. Risk Rating

Medium. Successful reflected JavaScript execution was observed, but no Stored or DOM XSS vulnerabilities were confirmed.

5. Recommendations

1. Apply context-aware output encoding.
2. Validate and sanitize user input.
3. Implement a Content Security Policy (CSP).
4. Avoid innerHTML and document.write().
5. Use textContent where appropriate.
6. Perform regular security testing.